[LAB-09] 7. [PBT] 선형회귀 파이프라인 - 추가예제 (1)

#01. 준비작업

1. 라이브러리 참조

```
from hossam import load_data
from helpers import *
```

```
📦 아이티윌 이광호 강사가 제작한 라이브러리를 사용중입니다.
📚 자세한 사용 방법은 https://py.hossam.kr 을 참고하세요.
📧 Email: leekh4232@gmail.com
🎬 Youtube: https://www.youtube.com/@hossam-codingclub
📝 Blog: https://blog.hossam.kr/
🩹 Version: develop
```

2. 데이터 가져오기와 타입 확정

```
origin = load_data("housing_qtcheck")
cat_desc = load_data("housing_categorical_summary")

# 명목형 기술통계량 표의 컬럼명이 곧 명목형 변수 목록이다
df = my_qtcheck.set_type(origin, as_category=cat_desc.columns)
```

```
📚 주택 가격 데이터셋의 품질 검사 완료 버전 (출처: 자체 정제)
📚 주택 가격 데이터셋의 범주형 변수에 대한 기술 통계량 (출처: 자체 작업)


<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype   
---  ------            --------------  -----   
 0   price             545 non-null    int64   
 1   area              545 non-null    int64   
 2   bedrooms          545 non-null    int64   
 3   bathrooms         545 non-null    int64   
 4   stories           545 non-null    int64   
 5   mainroad          545 non-null    category
 6   guestroom         545 non-null    category
 7   basement          545 non-null    category
 8   hotwaterheating   545 non-null    category
 9   airconditioning   545 non-null    category
 10  parking           545 non-null    int64   
 11  prefarea          545 non-null    category
 12  furnishingstatus  545 non-null    category
dtypes: category(7), int64(6)
memory usage: 30.3 KB
```

2. 변수 유형 분류

```
target = "price"

nominal_cols = my_qtcheck.get_categorical_column_names(df)
continuous_cols = my_qtcheck.get_number_column_names(df)
continuous_cols.remove(target)

print("종속변수 :", target)
print("명목형   :", nominal_cols)
print("연속형   :", continuous_cols)
```

```
종속변수 : price
명목형   : ['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
'prefarea', 'furnishingstatus']
연속형   : ['area', 'bedrooms', 'bathrooms', 'stories', 'parking']
```

#02. 기계적 분석

1. 전처리 체크포인트별 모델 적합

```
# 전처리를 하나씩 누적한 체크포인트 (아래로 갈수록 전처리가 하나 더 붙는다)
checkpoints = {
    "0_기준선":    dict(),
    "1_로그변환":   dict(log=True),
    "2_이상치대체": dict(log=True, outlier=True),
    "3_공선성제거": dict(log=True, outlier=True, vif=True),
    "4_스케일링":   dict(log=True, outlier=True, vif=True, scale=True),
}

fits = {}

for name in checkpoints:
    print(f"\n🔍 --- {name} ---")
    fits[name] = my_ols.fit_pipeline(df, target, nominal_cols=nominal_cols,
                                     name=name, verbose=False, **checkpoints[name])
```

```
🔍 --- 0_기준선 ---

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.7145)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1161)
유의하지 않은 독립변수 제거 → guestroom (p = 0.0532)

🔍 --- 1_로그변환 ---

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.6187)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.0850)

🔍 --- 2_이상치대체 ---

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.6357)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1060)
```

```
❓ --- 3_공선성제거 ---

완료! 남은 변수: ['area', 'bathrooms', 'bedrooms', 'parking', 'stories']
최대 VIF = 1.41

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.6357)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1060)

❓ --- 4_스케일링 ---

완료! 남은 변수: ['area', 'bathrooms', 'bedrooms', 'parking', 'stories']
최대 VIF = 1.41

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.6357)


유의하지 않은 독립변수 제거 → bedrooms (p = 0.1060)
```

2. 성능 비교와 모델 선택

```
final_fit = my_ols.compare_models(fits)

print("최종 모델 :", final_fit.name_)
print("독립변수  :", int(final_fit.df_model), "개")
```

모델	변수수	R2(모델척도)	Adj.R2	AIC	BIC	R2(원본척도)	RMSE	MAE	Gap(%)
0_기준선	10	0.677	0.671	16692.908	16740.216	0.677	1061796.015	778354.185	2.910
2_이상치대체	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
3_공선성제거	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
4_스케일링	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
1_로그변환	11	0.704	0.698	-171.957	-120.347	0.695	1032517.732	742613.119	0.080

```
최종 모델 : 0_기준선
독립변수  : 10 개
```

#03. 도메인 지식 반영

1. 편의 요소를 하나의 점수로

```
# 모두 "이 주택이 어떤 편의 요소를 갖췄는가"를 묻는 같은 성격의 yes/no 항목이다
amenity_cols = ["mainroad", "guestroom", "basement",
                "hotwaterheating", "airconditioning", "prefarea"]

df2 = df.copy()
df2["amenity"] = sum((df2[c] == "yes").astype(int) for c in amenity_cols)
df2 = df2.drop(columns=amenity_cols)
```

```
# 명목형 목록도 df2 기준으로 다시 잡는다
nominal_cols2 = [c for c in nominal_cols if c in df2.columns]

print("명목형 :", nominal_cols2)
print(df2["amenity"].value_counts().sort_index().to_string())
df2.head()
```

```
명목형 : ['furnishingstatus']
amenity
0     42
1    172
2    164
3    102
4     50
5     15
```

	price	area	bedrooms	bathrooms	stories	parking	furnishingstatus	amenity
0	13300000	7420	4	2	3	2	furnished	3
1	12250000	8960	4	4	4	3	furnished	2
2	12250000	9960	3	2	2	2	semi-furnished	3
3	12215000	7500	4	2	2	3	furnished	4
4	11410000	7420	4	1	2	2	furnished	4

2. 도메인 버전 모델 적합 (fits 에 누적)

```
domain_checkpoints = {
    "5_도메인_기준선":    dict(),
    "6_도메인_로그변환":   dict(log=True),
    "7_도메인_이상치대체": dict(log=True, outlier=True),
    "8_도메인_공선성제거": dict(log=True, outlier=True, vif=True),
}

for name in domain_checkpoints:
    print(f"\n▶ --- {name} ---")
    fits[name] = my_ols.fit_pipeline(df2, target, nominal_cols=nominal_cols2,
                                     name=name, verbose=False,
        **domain_checkpoints[name])

print("\n누적된 모델 :", len(fits), "개")
```

```
▶ --- 5_도메인_기준선 ---

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.6083)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1361)

▶ --- 6_도메인_로그변환 ---

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.7440)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.0947)

▶ --- 7_도메인_이상치대체 ---
```

```
유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.7595)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1152)

❓ --- 8_도메인_공선성제거 ---

완료! 남은 변수: ['amenity', 'area', 'bathrooms', 'bedrooms', 'parking', 'stories']
최대 VIF = 1.42

유의하지 않은 독립변수 제거 → furnishingstatus_1 (p = 0.7595)
유의하지 않은 독립변수 제거 → bedrooms (p = 0.1152)

누적된 모델 : 9 개
```

#03. 전체 모델 성능 비교

```python
final_fit = my_ols.compare_models(fits)

print("최종 모델 :", final_fit.name_)
print("독립변수  :", int(final_fit.df_model), "개")
```

모델	변수 수	R2(모델척도)	Adj.R2	AIC	BIC	R2(원본척도)	RMSE	MAE	Gap(%)
7_도메인_이상치대체	6	0.697	0.694	-168.348	-138.243	0.682	1054311.081	755397.826	2.190
8_도메인_공선성제거	6	0.697	0.694	-168.348	-138.243	0.682	1054311.081	755397.826	2.190
6_도메인_로그변환	6	0.696	0.693	-167.034	-136.928	0.681	1055310.409	756568.181	2.280
5_도메인_기준선	6	0.669	0.665	16698.145	16728.250	0.669	1074769.154	781098.845	4.170
0_기준선	10	0.677	0.671	16692.908	16740.216	0.677	1061796.015	778354.185	2.910
2_이상치대체	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
3_공선성제거	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
4_스케일링	11	0.705	0.699	-173.145	-121.536	0.695	1031742.643	741132.622	0.000
1_로그변환	11	0.704	0.698	-171.957	-120.347	0.695	1032517.732	742613.119	0.080

```
최종 모델 : 7_도메인_이상치대체
독립변수  : 6 개
```